

1 слайд (титульный)

Добрый день, уважаемая комиссия и слушатели!

Меня зовут Иван Добрынин. Тема моей выпускной квалификационной работы: Разработка комплекса информационных систем для проведения соревнований и тренировок по спортивному программированию в формате дуэли.

2 слайд (введение в предметную область)

Спортивное программирование приобретает всё большую популярность. Общепринятым форматом проведения соревнований является контест, в котором может быть произвольное количество участников. Контест содержит набор задач, на решение которых выделяется несколько часов.

Для решения задачи требуется написать код на одном из допустимых языков программирования, который будет обрабатывать входные данные и выдавать результат в соответствии с условием задачи.

Формат дуэли представляет собой соревнование между двумя участниками, которые одновременно решают одну и ту же задачу и стараются сделать это быстрее друг друга.

3 слайд (сравнение существующих платформ)

Существуют различные платформы для решения задач по спортивному программированию, такие как Codeforces, LeetCode и Codebattle. На слайде отображено функциональное сравнение этих платформ по ряду критериев.

Большинство платформ ориентировано на проведение классических контестов, и не поддерживают проведение дуэлей и гибкую настройку структуры турниров.

Использование своих задач плюс-минус поддерживают все три платформы, однако они не приспособлены для нестандартных типов задач.

Также в эпоху развития искусственного интеллекта остро встаёт вопрос проверки честности участия в соревнованиях, причем анализу должен подвергаться именно процесс написания кода, а не конечный результат.

4 слайд (постановка задачи)

Для выполнения данной работы требуется спроектировать архитектуру платформы, написать и протестировать код всех компонентов системы, а также разработать масштабируемую тестирующую систему, которая будет автоматически проверять решения пользователей с поддержкой параллельного исполнения и эффективным использованием вычислительных ресурсов. Более того, требуется исследовать подход к обнаружению подозрительных участников на основе анализа их поведения.

5 слайд (микросервисная архитектура системы)

Для реализации требований была спроектирована распределенная микросервисная архитектура, которая состоит из нескольких основных и инфраструктурных компонентов.

Выделение компонентов в независимые сервисы было сделано с целью разделения ответственности между ними и обеспечения возможности масштабирования отдельных частей системы.

Интерфейс взаимодействия с пользователем представляет собой браузерное приложение, разработанное с использованием React JS. Серверная часть платформы содержит три основных компонента:

- Duely — приложение на C#, в котором реализуется бизнес-логика, касающаяся пользователей, дуэлей и турниров.
- Taski — приложение на Go, которое предназначено для хранения задач и определения правил автоматического тестирования решений.
- Exesh — приложение на Go, используемое в качестве «песочницы» для запуска произвольных команд, требуемых для тестирования решений.

В платформе есть еще два вспомогательных компонента:

- Analyzer — приложение на Python, предназначенное для анализа поведения пользователей во время участия в дуэли.
- Dashboard — приложение на Python, с помощью которого происходит визуализация и мониторинг работы Exesh в режиме реального времени.

6 слайд (схема асинхронного процесса тестирования)

Тестирование решения задачи представляет собой запуск решения на наборе тестов. Этот процесс может занимать достаточно продолжительное время, поэтому он построен в асинхронной парадигме.

Пользователь инициирует отправку решения на проверку, после чего процесс последовательно проходит через Duely, Taski и Exesh. Далее по мере выполнения команд тестирования происходит асинхронное распространение статусов в обратном порядке с использованием брокера обмена сообщениями.

7 слайд (распределенное исполнение графа команд)

Процесс тестирования решения может быть представлен в виде ориентированного ациклического графа, вершинами которого являются атомарные команды (например, компиляция кода или запуск на одном тесте). В терминах системы граф называется Execution, а вершина — Job.

Исполнение этого графа можно выполнять параллельно на независимых узлах системы. В качестве примера на слайде изображено распределение джобов по двум узлам.

8 слайд (архитектура Exesh на основе модели оркестрации)

Внутренняя архитектура Exesh включает в себя два вида компонентов, которые образуют модель оркестрации. Центральным компонентом является Coordinator, который управляет исполнением графов, а за непосредственный запуск джобов отвечает Worker.

9 слайд (хранение и передача файлов между компонентами системы)

Тестирование задачи включает в себя достаточно плотное взаимодействие с файловой системой. Для этого в рамках работы была реализована библиотека filestorage, которая предоставляет удобную абстракцию над файловой системой, кэширование для повторного использования, передачу файлов между компонентами в сжатом виде и автоматическую очистку неиспользуемых ресурсов.

10 слайд (честное распределение ресурсов для одновременного исполнения набора Execution)

Для реализации алгоритма планировщика представим, что исполнение джобов происходит последовательно. Пусть в текущий момент времени есть три Execution, в каждом из которых исполнен некоторый префикс джобов. Вопрос: какой из трех возможных джобов нужно исполнить следующим?

11 слайд (честное распределение ресурсов для одновременного исполнения набора Execution)

Ответ — тот, у которого больше приоритет в текущий момент времени.

По статистическим данным мы можем вычислить ожидаемое время выполнения каждого джоба. Приоритет Execution зависит от реальной продолжительности его исполнения, а также от ожидаемой оставшейся продолжительности исполнения. Интуиция такая: хотим, чтобы распределение было плюс-минус равномерное, но еще и чтобы более старые Execution получали больше ресурсов, чем новые.

12 слайд (распределение Jobs по исполнителям для эффективном утилизации ресурсов)

Джоб можно представить в виде прямоугольника с шириной, равной продолжительности выполнения, и высотой, равной максимальным потреблением оперативной памяти.

Worker в любой момент времени может исполнять не более некоторого количества джобов (на слайде, например, 4) и имеет ограничение на суммарное потребление оперативной памяти.

Пусть в текущий момент времени для исполнения доступен приоритетный красный джоб, которому требуется много памяти, и три неприоритетных желтых джоба, которым требуется мало памяти. В текущий момент времени недостаточно ресурсов, чтобы взять на исполнение красный джоб, значит, по предыдущему алгоритму мы не будем ничего делать, пока не сможем взять его.

13 слайд (распределение Jobs по исполнителям для эффективном утилизации ресурсов)

На левой картинке изображено, что будет в таком случае. На правой картинке проиллюстрирован разработанный promise-based алгоритм, заполняющий пропуск желтыми джобами. Он вычисляет минимальное возможное время начала исполнения красного джоба и запоминает его. Далее он может работать с желтыми джобами до тех пор пока не ухудшается обещанное время начала исполнения красного.

14 слайд (безопасный запуск пользовательского кода в изолированном окружении)

Для безопасного запуска пользовательского кода была выбрана существующая утилита `isolate`, использующаяся на международной олимпиаде школьников по информатике. Утилита предоставляет возможность изоляции процессов друг от друга, ограничение выделяемых ресурсов, запрет взаимодействия с сетью, а также имеет легкий вес и высокую производительность по сравнению с использованием Docker-контейнеров.

15 слайд (анализ подозрительности поведения участника при участии в дуэли)

Для проверки честности участников система анализирует не конечный код пользователя, а его поведение при участии в дуэли. В рамках работы были выявлены поведенческие паттерны во время написания кода самостоятельно и при помощи сторонних инструментов генерации кода. В процессе участия в дуэли собираются события, относящиеся к написанию кода и взаимодействию с интерфейсом системы.

16 слайд (анализ подозрительности поведения участника при участии в дуэли)

После окончания дуэли список событий каждого пользователя трансформируется в вектор признаков. Особое внимание уделено следующим параметрам:

- Скорость ввода кода и ее разброс. Если код появляется слишком равномерно, то это вызывает подозрение.
- Вставки большого количества кода, а также вставки без последующего редактирования вставленного фрагмента являются маркерами подозрительности.
- Количество значимых перемещений курсора. В процессе наблюдения за собственным написанием кода было выявлено, что нередко происходит возврат к ранее написанным участкам кода и их изменение.

По вектору признаков происходит моделирование подозрительности поведения пользователя — получаем число от 0 до 1.

В качестве моделей были выбраны логистическая регрессия и метод случайного леса. Логистическая регрессия имеет высокую интерпретируемость и малый риск переобучения. Она позволит получить достаточно стабильную базовую оценку. Метод случайного леса способен учитывать сложные нелинейные зависимости между признаками, которые характерны для поведенческих данных пользователей.

17 слайд (анализ подозрительности поведения участника при участии в дуэли)

На слайде представлены метрики качества работы моделей. Несмотря на маленький объем обучающей выборки, обе модели показали достаточно хорошие результаты. Можно заметить достаточно высокое значение false positive rate, однако это лишь значит, что модели слишком подозрительны и чаще подозревают честных пользователей. При этом также имеем большой recall — это значит, что модели не пропускают действительно нечестных пользователей, пусть и взамен на излишнюю подозрительность.

Сбор обучающей выборки производился вручную мной, а также был проведен турнир на занятии кружка по олимпиадному программированию на нашем факультете.

18 слайд (настройка правил проведение дуэлей и турниров)

Кстати, скрин результатов турнира из интерфейса платформы можно увидеть на правой картинке. На левой картинке отображена гибкая настройка правил дуэли. Можно выбрать, показывать ли код оппонентов друг другу во время дуэли, а также количество и темы задач.

19 слайд (результаты работы)

Подводя итоги работы хочется сказать, что одним из главных достижений, по моему мнению, является то, что платформа целиком была проверена в реальных условиях проведения турнира с реальными пользователями. Была проделана огромная работа по разработке распределенной тестирующей системы и алгоритмов планировщиков, а также было проведено большое исследование оценки подозрительности поведения пользователей во время дуэли.